TP Flotte de véhicules de service

Contexte

Une entreprise met à disposition de ses salariés une **flotte de véhicules de service** pour leurs déplacements professionnels. Aujourd'hui, les réservations sont notées dans un tableur : on ne sait pas toujours qui a pris quel véhicule, ni quand il doit revenir.

On veut écrire une application Java qui permet au gestionnaire de flotte de suivre les véhicules, les salariés et les réservations. L'application est organisée selon le modèle **MVC** : les données sont stockées dans une base **MySQL** via une couche **DAO**, et toutes les opérations passent par des **contrôleurs**. Les contrôleurs s'appuient sur des services, qui contiennent les règles de gestion. La vue (menu console) ne parle jamais directement à la base.

Objectifs pédagogiques

- Séparer les responsabilités : modèle, persistance (DAO), service, contrôleur, vue.
- Accéder à une base MySQL avec JDBC et des requêtes préparées.
- Utiliser des `Comparator`, des collections et une classe générique.
- Exécuter des traitements longs dans des threads.
- Chiffrer des données avec AES.
- Tracer l'activité avec log4j et documenter le code avec la JavaDoc.

Prérequis techniques

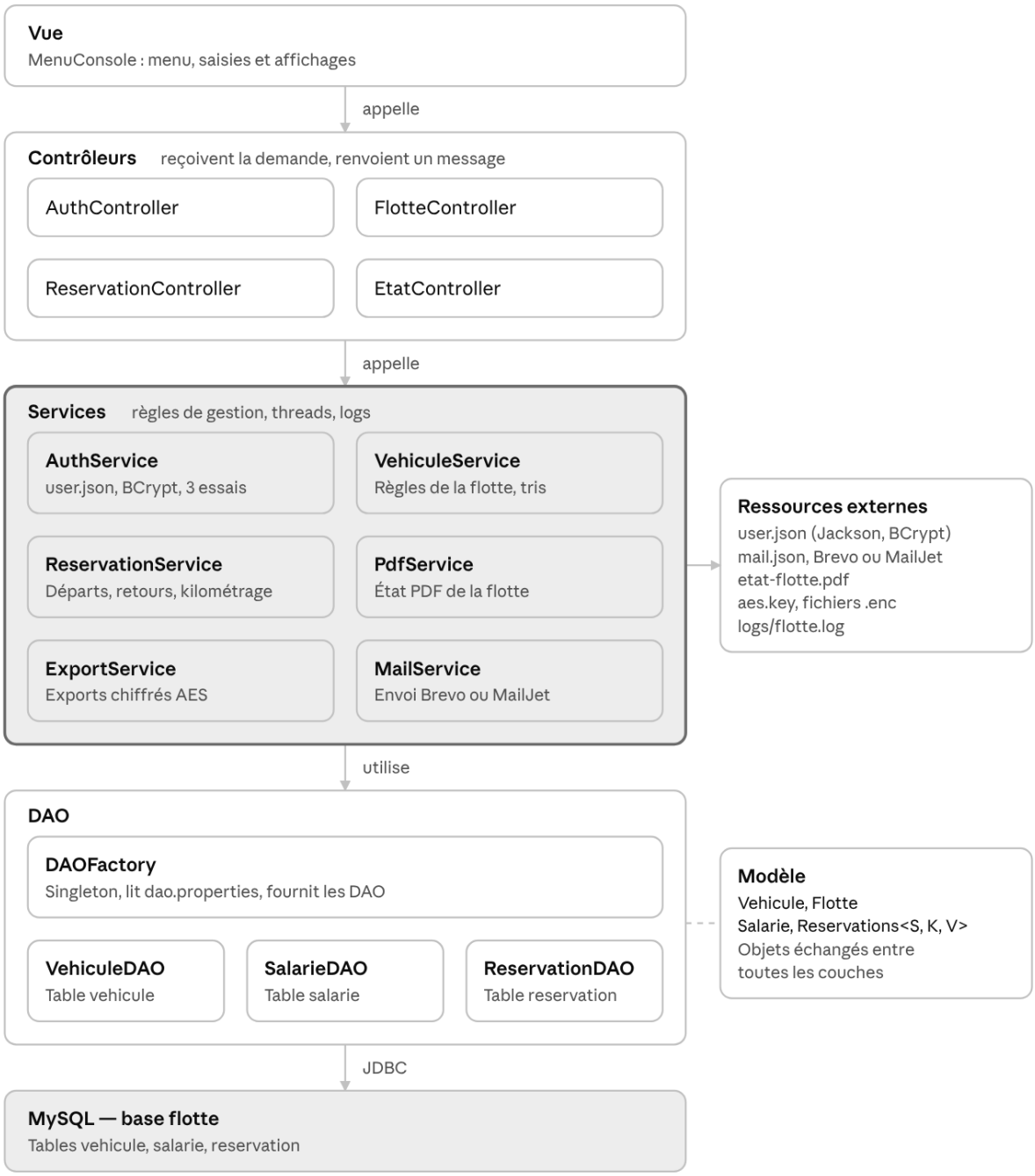
Besoin	Librairie conseillée
Accès à MySQL	MySQL Connector/J (`com.mysql:mysql-connector-j`)
Journalisation	Apache Log4j 2
Lecture des fichiers JSON	Jackson Databind
Hachage du mot de passe	jBCrypt (`org.mindrot:jbcrypt`)
Génération du PDF	OpenPDF ou iText
Envoi de courriel	API Brevo ou MailJet
Chiffrement AES	Inclus dans le JDK (`javax.crypto`)

Un serveur MySQL local (XAMPP, WAMP, MAMP ou Docker) doit être disponible. Le projet est géré avec Maven de préférence.

Architecture attendue

L'application est découpée en packages : `vue`, `controleur`, `service`, `dao`, `modele` et `util`. Chaque couche n'appelle que celle du dessous : vue → contrôleur → service → DAO → MySQL.

Les contrôleurs appellent les services ; seuls les DAO accèdent à MySQL



architecture MVC + services + DAO · 5 couches

La vue transmet chaque action à un contrôleur. Le contrôleur **appelle un service**, qui applique les règles de gestion, lance les threads, écrit les logs et délègue la persistance aux DAO.

Règles à respecter

- La vue n'appelle que les contrôleurs.
- Les contrôleurs n'appellent que les services : jamais de DAO ni de `java.sql` dans un contrôleur.
- Les règles de gestion sont écrites uniquement dans les services.
- Aucune requête SQL n'est écrite en dehors du package `dao`.
- Le modèle ne dépend d'aucune autre couche.

Pourquoi une couche service ?

L'idée. Pensez à un restaurant. Le **serveur** (le contrôleur) prend la commande et rapporte l'assiette. Le **cuisinier** (le service) prépare le plat en suivant la recette. Le **magasinier** (le DAO) va chercher les ingrédients dans la réserve (la base). Le serveur ne cuisine pas, et le cuisinier ne va pas en salle.

Sans service, toute la « recette » se retrouve dans le contrôleur : il lit la saisie, vérifie les règles, appelle plusieurs DAO, lance les threads. Il devient long, difficile à lire et impossible à réutiliser.

Ce que la couche service apporte.

1. **Les règles de gestion sont au même endroit.** « Un véhicule déjà en déplacement ne peut pas repartir » est écrit une seule fois, dans `ReservationService`.
2. **Les contrôleurs restent courts.** Ils reçoivent la demande, appellent le service et renvoient un message à la vue.
3. **Le code est réutilisable.** Si demain le menu console est remplacé par une interface JavaFX ou une application web, on change la vue et les contrôleurs ; les services sont gardés tels quels.
4. **Une action peut utiliser plusieurs DAO.** Enregistrer un retour met à jour la réservation **et** le kilométrage du véhicule : c'est le service qui coordonne les deux.
5. **Les tests sont plus simples.** On peut tester un service sans clavier ni menu, en appelant directement ses méthodes.

Qui fait quoi, sur un exemple : le retour d'un véhicule.

Couche	Rôle	Ce qu'elle fait pour un retour
Vue	Lire et afficher	Demande l'immatriculation et le kilométrage, affiche le message reçu
Contrôleur	Recevoir la demande, appeler le service	Appelle `enregistrerRetour(...)` et transforme le résultat en message
Service	Appliquer les règles de gestion	Vérifie que le véhicule est en déplacement et que le kilométrage est cohérent, puis appelle les DAO
DAO	Exécuter le SQL	`UPDATE reservation ...` et `UPDATE vehicule ...`

Le contrôleur appelle le service.

```
package controleur;

public class ReservationController {
```

```
    private final ReservationService reservationService;

    public ReservationController(ReservationService reservationService) {
        this.reservationService = reservationService;
    }

    /** Renvoie le message à afficher par la vue. */
    public String enregistrerRetour(String immatriculation, int kmRetour) {
        try {
            reservationService.enregistrerRetour(immatriculation, kmRetour);
            return "Retour enregistré.";
        } catch (RegleGestionException e) {
            return "Refusé : " + e.getMessage();
        }
    }
}
```

Le service applique les règles et utilise les DAO.

```
package service;

public class ReservationService {

    private static final Logger log = LogManager.getLogger(ReservationService.class);

    private final VehiculeDAO vehiculeDAO;
    private final ReservationDAO reservationDAO;

    public ReservationService(VehiculeDAO vehiculeDAO, ReservationDAO reservationDAO)
{
        this.vehiculeDAO = vehiculeDAO;
        this.reservationDAO = reservationDAO;
    }

    public void enregistrerRetour(String immatriculation, int kmRetour) {
        Vehicule v = vehiculeDAO.trouverParImmatriculation(immatriculation);

        // Règles de gestion
        if (v == null) {
            throw new RegleGestionException("véhicule inconnu");
        }
        if (!reservationDAO.estEnDeplacement(v.getId())) {
            throw new RegleGestionException("ce véhicule n'est pas en déplacement");
        }
        if (kmRetour < v.getKilometrage()) {
            throw new RegleGestionException("kilométrage inférieur au compteur
actuel");
        }

        // Persistance : deux DAO pour une seule action
        reservationDAO.cloturer(v.getId(), new Timestamp(System.currentTimeMillis()),
kmRetour);
        v.setKilometrage(kmRetour);
        vehiculeDAO.modifier(v);
```

```
        log.info("Retour de {} à {} km", immatriculation, kmRetour);
    }
}
```

`RegleGestionException` est une exception non vérifiée du package `service`. Elle signale qu'une règle n'est pas respectée ; son message est affiché tel quel à l'utilisateur.

Qui crée les objets ? La classe `Main` assemble l'application une seule fois au démarrage, de la base vers la vue :

```
DAOFactory factory = DAOFactory.getInstance();

ReservationService reservationService =
        new ReservationService(factory.getVehiculeDAO(),
factory.getReservationDAO());
ReservationController reservationController = new
ReservationController(reservationService);

new MenuConsole(reservationController /* , autres contrôleurs */).demarrer();
```

Question 1 — Le modèle `Vehicule`

Écrivez la classe `Vehicule` du package `modele`, qui représente un véhicule de service avec :

- un identifiant `id` (entier, attribué par la base) ;
- l'immatriculation, au format `AA-123-AA` ;
- la marque et le modèle ;
- la date de mise en circulation (`LocalDate`) ;
- le kilométrage actuel (entier, en km).

Prévoyez les constructeurs, accesseurs et une méthode `toString()` qui affiche les informations du véhicule. Le format de l'immatriculation et un kilométrage positif sont vérifiés à la création ; en cas d'erreur, une `IllegalArgumentException` est levée.

La classe ne contient **aucun** code d'accès à la base ni de saisie clavier : la saisie est faite par la vue, qui transmet les valeurs au contrôleur.

Question 2 — Base de données et couche DAO

2.1 — Création de la base. Écrivez et exécutez le script `flotte.sql` :

```
CREATE DATABASE IF NOT EXISTS flotte CHARACTER SET utf8mb4;
USE flotte;

CREATE TABLE vehicule (
    id                   INT AUTO_INCREMENT PRIMARY KEY,
    immatriculation      CHAR(9)      NOT NULL UNIQUE,
    marque               VARCHAR(50)  NOT NULL,
    modele               VARCHAR(50)  NOT NULL,
    date_mise_circulation DATE        NOT NULL,
    kilometrage          INT          NOT NULL DEFAULT 0
);

CREATE TABLE salarie (
```

```
    matricule CHAR(6)      PRIMARY KEY,
    nom       VARCHAR(100) NOT NULL,
    prenom    VARCHAR(100) NOT NULL
);

CREATE TABLE reservation (
    id             INT AUTO_INCREMENT PRIMARY KEY,
    matricule      CHAR(6)   NOT NULL,
    id_vehicule    INT       NOT NULL,
    date_depart    TIMESTAMP NOT NULL,
    date_retour    TIMESTAMP NULL,
    km_retour      INT       NULL,
    FOREIGN KEY (matricule)   REFERENCES salarie(matricule),
    FOREIGN KEY (id_vehicule) REFERENCES vehicule(id)
);
```

2.2 — Paramètres de connexion. Les paramètres ne doivent pas être écrits dans le code. Placez-les dans `src/main/resources/dao.properties` :

```
url=jdbc:mysql://127.0.0.1:3306/flotte
driver=com.mysql.cj.jdbc.Driver
user=root
password=
```

2.3 — La fabrique de DAO. Écrivez la classe `DAOFactory` du package `dao` :

- c'est un **singleton** : constructeur privé, attribut `static instance`, méthode `getInstance()` qui crée l'instance une seule fois ;
- elle lit `constantes.properties`;
- elle fournit `getConnection()` ainsi que `getVehiculeDAO()`, `getSalarieDAO()` et `getReservationDAO()`.

Les erreurs techniques (SQL, configuration) sont encapsulées dans une exception non vérifiée `DAOException`.

2.4 — Le DAO des véhicules. Définissez l'interface `VehiculeDAO`, puis son implémentation `VehiculeDAOImpl` :

```
public interface VehiculeDAO {
    void creer(Vehicule vehicule);                    // renseigne l'id généré
    Vehicule trouverParId(int id);
    Vehicule trouverParImmatriculation(String immat);
    List<Vehicule> lister();
    void modifier(Vehicule vehicule);                 // ex. : mise à jour du
kilométrage
    void supprimer(int id);
}
```

Toutes les requêtes utilisent des `PreparedStatement` (jamais de concaténation de chaînes) et des blocs try-with-resources pour fermer connexions, requêtes et résultats.

Question 3 — La flotte, son service et son contrôleur

3.1 — Le modèle `Flotte`. Une flotte est une collection de véhicules (`List<Vehicule>`) représentés par la classe de la question 1. Elle sert de copie en mémoire des données de la base.

3.2 — Le service. Écrivez la classe `VehiculeService` du package `service`. Elle reçoit un `VehiculeDAO`, un `ReservationDAO` (pour vérifier les réservations en cours) et une `Flotte`, et contient la logique des opérations :

- ajouter un véhicule ;
- afficher la flotte ;
- retirer un véhicule de la flotte (impossible s'il a des réservations en cours) ;
- rechercher un véhicule par son immatriculation ;
- trier la flotte par marque, puis par modèle ;
- trier la flotte par date de mise en circulation (du plus ancien au plus récent) ;
- trier la flotte par kilométrage décroissant.

Chaque opération qui modifie les données vérifie d'abord les règles de gestion (immatriculation unique, pas de réservation en cours avant un retrait), passe ensuite par le DAO, puis met à jour la liste en mémoire. Si une règle n'est pas respectée, le service lève une `RegleGestionException`. Les tris utilisent des `Comparator` (annexe A), par exemple :

```
Comparator<Vehicule> parMarqueModele = Comparator.comparing(Vehicule::getMarque)
                                                 .thenComparing(Vehicule::getModele);
Comparator<Vehicule> parDate =
Comparator.comparing(Vehicule::getDateMiseCirculation);
Comparator<Vehicule> parKmDecroissant =
Comparator.comparingInt(Vehicule::getKilometrage)
                                                  .reversed();
```

3.3 — Le contrôleur. Écrivez la classe `FlotteController` du package `controleur`. Elle reçoit un `VehiculeService` et propose une méthode par opération du menu. Chaque méthode appelle le service, puis renvoie à la vue le résultat ou un message (succès, ou erreur issue d'une `RegleGestionException`). Le contrôleur ne contient **aucune** règle de gestion et n'utilise **aucun** DAO.

3.4 — La vue. La classe `MenuConsole` du package `vue` affiche le menu, lit les saisies avec un `Scanner` et appelle **uniquement** les méthodes des contrôleurs.

3.5 — Traces. Les services tracent toutes ces opérations avec log4j dans le fichier `logs/flotte.log`. Les erreurs remontées par les DAO sont journalisées au niveau `ERROR`.

Question 4 — Authentification

Seul le gestionnaire de flotte peut utiliser l'application. Au lancement, `AuthController` récupère le login et le mot de passe saisis dans la vue, puis appelle `AuthService`, qui fait la vérification.

Le login et le mot de passe haché avec BCrypt sont stockés dans le fichier `user.json`, lu par `AuthService` avec la librairie Jackson :

```
{
  "login": "gestionnaire",
  "motDePasseHash": "$2a$10$NtvCRBsXncSgwinwGcjDge9QigBYqbR6MaJlY5kqch4ODYFP0z2Oi"
}
```

Le mot de passe saisi est comparé au hash avec `BCrypt.checkpw(...)`. Après 3 échecs consécutifs, l'application s'arrête : c'est `AuthService` qui compte les échecs. Chaque tentative d'authentification, réussie ou non, est tracée dans le fichier log.

Question 5 — Chargement et sauvegarde en base

La persistance de la flotte se fait dans MySQL, par l'intermédiaire des DAO.

- **À l'ouverture**, `VehiculeService` charge tous les véhicules avec `VehiculeDAO.lister()` pour alimenter la flotte en mémoire.
- **La sauvegarde** (ajout, modification, suppression en base) s'exécute dans un **thread** séparé, pour ne pas bloquer le menu. Chaque service qui écrit en base utilise un `ExecutorService` à un seul thread (annexe B).
- Chaque service propose une méthode `fermer()` qui arrête proprement son `ExecutorService` (`shutdown()` puis `awaitTermination(...)`). À la fermeture de l'application, `Main` appelle `fermer()` sur chaque service, pour que toutes les écritures soient terminées.

Question 6 — État PDF de la flotte

On souhaite produire un état de la flotte au format PDF, avec pour chaque véhicule : immatriculation, marque, modèle, date de mise en circulation, kilométrage et disponibilité (disponible ou en déplacement).

La classe `PdfService` du package `service` récupère les données via les DAO, puis génère le fichier `etat-flotte.pdf` avec OpenPDF ou iText. La génération s'exécute dans **un autre thread**, et sa fin est tracée dans le log. `EtatController` se contente d'appeler `PdfService` et d'afficher un message de confirmation.

Question 7 — Envoi de l'état par courriel

L'état PDF est envoyé en pièce jointe à `pascal.lamy5@ac-rennes.fr` avec l'API **Brevo** ou **MailJet**. L'envoi est réalisé par la classe `MailService`, dans un **thread** ; `EtatController` l'appelle après la génération du PDF.

Le contenu du courriel est stocké dans le fichier `mail.json`, lu par `MailService` avec Jackson :

```
{
  "destinataire": "pascal.lamy5@ac-rennes.fr",
  "sujet": "État de la flotte de véhicules",
  "corps": "Veuillez trouver ci-joint l'état de la flotte de véhicules de service."
}
```

La clé d'API n'est jamais écrite dans le code : elle est lue dans un fichier de configuration non versionné (par exemple `mail.properties`).

Question 8 — Salariés et réservations

Chaque salarié est identifié par un **matricule à 6 chiffres**. On souhaite gérer les réservations de véhicules par les salariés.

8.1 — Le modèle.

- Créez la classe `Salarie` (nom, prénom, matricule). Le matricule est validé à la création (expression régulière `\d{6}`).

- Toutes les réservations passées et en cours d'un salarié sont enregistrées dans une `HashMap` : la clé est la date de départ au format `Timestamp`, la valeur est le véhicule.
- Créez une **classe générique à 3 paramètres** qui gère l'ensemble des réservations d'un salarié, par exemple `Reservations<S, K, V>` instanciée en `Reservations<Salarie, Timestamp, Vehicule>`.
- Cette structure est insérée dans une collection qui regroupe les réservations de tous les salariés.

8.2 — La persistance. Écrivez `SalarieDAO` et `ReservationDAO` (interfaces et implémentations) qui travaillent sur les tables `salarie` et `reservation` créées à la question 2. `ReservationDAO` propose notamment `estEnDeplacement(int idVehicule)` et `cloturer(int idVehicule, Timestamp dateRetour, int kmRetour)`. Au démarrage, la collection des réservations est reconstruite à partir de la base.

8.3 — Le service et le contrôleur. La classe `ReservationService` contient la logique des opérations ci-dessous ; la classe `ReservationController` les propose au menu en appelant ce service :

- créer un salarié ;
- enregistrer le **départ** d'un véhicule avec un salarié ;
- enregistrer le **retour** d'un véhicule, avec le kilométrage relevé au compteur ;
- afficher l'historique des réservations d'un salarié ;
- afficher les véhicules actuellement en déplacement.

8.4 — Règles de gestion, écrites dans `ReservationService`.

- Un véhicule déjà en déplacement ne peut pas partir à nouveau tant que son retour n'est pas enregistré.
- Au retour, le kilométrage saisi doit être supérieur ou égal au kilométrage actuel du véhicule. Le kilométrage du véhicule est alors mis à jour en base.
- Un salarié ne peut avoir qu'un seul véhicule en déplacement à la fois.

Une règle non respectée lève une `RegleGestionException`, dont le message est affiché par la vue. Toutes ces opérations sont tracées dans le log par le service. L'enregistrement d'un retour est donné en exemple complet dans la partie « Pourquoi une couche service ? ».

Question 9 — Export chiffré des salariés et des réservations

On souhaite exporter la liste des salariés et la liste des réservations dans des fichiers texte **chiffrés avec AES**, pour qu'ils ne soient pas lisibles sans la clé. Les exports sont réalisés par la classe `ExportService` du package `service` ; `EtatController` l'appelle depuis le menu.

9.1 — La classe utilitaire `ChiffrementAES`. Écrivez cette classe dans un package `util`. Elle propose :

- `genererCle()`, `sauvegarderCle(...)` et `chargerCle(...)` : la clé AES est générée au premier lancement, puis enregistrée en Base64 dans le fichier `aes.key` ;
- `chiffrer(String texte, SecretKey cle, Path fichier)` : chiffre un texte et l'écrit dans un fichier ;
- `dechiffrer(SecretKey cle, Path fichier)` : relit le fichier et renvoie le texte en clair.

Utilisez l'algorithme `AES/GCM/NoPadding` avec un vecteur d'initialisation (IV) aléatoire, différent à chaque export. L'annexe C donne un exemple complet.

9.2 — Export des salariés. Récupérez les salariés avec `SalarieDAO`, construisez un texte avec une ligne par salarié, puis chiffrez-le dans `salaries.enc` :

```
matricule;nom;prenom
204518;Le Goff;Morgane
```

9.3 — Export des réservations. Récupérez les réservations avec `ReservationDAO`, construisez un texte avec une ligne par réservation, puis chiffrez-le dans `reservations.enc` :

```
matricule;immatriculation;date_depart;date_retour;km_retour
204518;GH-482-KL;2026-10-20 08:15:00;2026-10-20 17:40:00;48210
204518;AB-123-CD;2026-10-27 09:00:00;;
```

Une date de retour vide signifie que le véhicule est encore en déplacement.

9.4 — Exécution et vérification.

- Chaque export s'exécute dans un thread, via l'`ExecutorService` de `ExportService` (annexe B).
- Ajoutez au menu une option qui déchiffre un fichier exporté et affiche son contenu, pour vérifier que l'export est correct.
- Chaque export est tracé dans le log (nom du fichier, nombre de lignes). Le contenu en clair n'est jamais écrit dans le log.
- Le fichier `aes.key` ne doit pas être versionné ni remis dans l'archive : sans lui, les exports sont illisibles.

Question 10 — JavaDoc

Toutes les classes et méthodes publiques sont documentées avec des commentaires JavaDoc (`@param`, `@return`, `@throws`). Générez la documentation dans le dossier `doc` du projet, avec `javadoc -d doc ...` ou le plugin Maven Javadoc.

Livrables

Vous remettrez votre travail sur Moodle sous la forme d'une archive **zip** du projet, contenant :

- ☐ les sources Java, organisées selon les packages `modele`, `dao`, `service`, `controleur`, `vue` et `util` ;
- ☐ le fichier `pom.xml` (ou la liste des librairies utilisées) ;
- ☐ le script `flotte.sql` de création de la base, avec quelques véhicules et salariés de test ;
- ☐ un fichier `dao.properties.example` sans le vrai mot de passe ;
- ☐ les fichiers `user.json` et `mail.json` ;
- ☐ le fichier de configuration `log4j2.xml` ;
- ☐ le dossier `doc` contenant la JavaDoc ;
- ☐ un court `README` expliquant comment créer la base et lancer l'application.

Aucune clé d'API, aucune clé AES (`aes.key`) ni aucun mot de passe réel ne doit figurer dans l'archive.

Annexe A — Trier avec un `Comparator`

L'idée. Pour trier une liste, Java doit savoir, pour deux véhicules donnés, lequel vient en premier. Un `Comparator` est simplement cette **règle de comparaison**. On la donne à `sort(...)`, qui s'occupe du reste.

Comment Java lit la réponse. La méthode `compare(a, b)` renvoie un entier :

Résultat de `compare(a, b)`	Signification
négatif (ex. -1)	`a` passe **avant** `b`
0	`a` et `b` sont équivalents
positif (ex. 1)	`a` passe **après** `b`

Trois façons d'écrire la même règle, de la plus longue à la plus courte :

```
// 1. Classe anonyme : tout est écrit
Comparator<Vehicule> parMarque = new Comparator<Vehicule>() {
    @Override
    public int compare(Vehicule a, Vehicule b) {
        return a.getMarque().compareTo(b.getMarque());
    }
};

// 2. Lambda : même chose, en plus court
Comparator<Vehicule> parMarque = (a, b) -> a.getMarque().compareTo(b.getMarque());

// 3. Comparator.comparing : « compare les véhicules selon leur marque »
Comparator<Vehicule> parMarque = Comparator.comparing(Vehicule::getMarque);
```

Utilisation.

```
vehicules.sort(parMarque);                          // A → Z
vehicules.sort(parMarque.reversed());               // Z → A
vehicules.sort(Comparator.comparing(Vehicule::getMarque)
                         .thenComparing(Vehicule::getModele));  // marque, puis
modèle
vehicules.sort(Comparator.comparingInt(Vehicule::getKilometrage)
                         .reversed());              // le plus roulé en premier
```

Pièges fréquents.

- `"renault".compareTo("Peugeot")` ne donne pas l'ordre attendu : les majuscules passent avant les minuscules. Pour ignorer la casse, utilisez `Comparator.comparing(Vehicule::getMarque, String.CASE_INSENSITIVE_ORDER)`.
- Pour un nombre (`int`), préférez `comparingInt(...)` : c'est plus rapide et on évite d'écrire `a - b`, qui peut déborder.
- Si une valeur peut être `null`, le tri plante avec une `NullPointerException`. Utilisez `Comparator.nullsLast(Comparator.naturalOrder())` pour placer ces véhicules à la fin.
- Un tri modifie l'ordre de la liste en mémoire, pas celui de la base. Pour trier dans la base, on utiliserait `ORDER BY` dans la requête SQL.

Annexe B — Lancer des tâches en arrière-plan avec `ExecutorService`

L'idée. Imaginez un guichet avec un employé. Vous déposez des dossiers (les **tâches**), l'employé les traite un par un pendant que vous continuez votre travail. Un `ExecutorService` est ce guichet : le programme principal (le menu) dépose des tâches et reste disponible pour le gestionnaire.

Pourquoi pas `new Thread(...)` ? Créer un thread à la main à chaque sauvegarde fonctionne, mais on ne contrôle ni leur nombre ni leur ordre, et on ne sait pas quand ils ont fini. L'`ExecutorService` réutilise ses threads, garde les tâches dans une file et s'arrête proprement.

Les 3 étapes.

1. **Créer** le service d'exécution, une seule fois, par exemple comme attribut d'une classe du package `service`.
2. **Soumettre** des tâches avec `submit(...)`.
3. **Arrêter** le service d'exécution à la fermeture de l'application.

```
public class VehiculeService {

    private static final Logger log = LogManager.getLogger(VehiculeService.class);

    // 1. Créer : un seul thread, donc les tâches s'exécutent dans l'ordre
    private final ExecutorService executor = Executors.newSingleThreadExecutor();

    public void ajouterVehicule(Vehicule vehicule) {
        flotte.ajouter(vehicule);                   // mise à jour immédiate en
mémoire
        // 2. Soumettre : l'écriture en base se fait en arrière-plan
        executor.submit(() -> {
            try {
                vehiculeDAO.creer(vehicule);
                log.info("Véhicule enregistré : {}", vehicule.getImmatriculation());
            } catch (DAOException e) {
                log.error("Échec de l'enregistrement", e);
            }
        });
    }

    // 3. Arrêter : appelé par Main quand le gestionnaire quitte l'application
    public void fermer() throws InterruptedException {
        executor.shutdown();                         // n'accepte plus de nouvelles
tâches
        if (!executor.awaitTermination(10, TimeUnit.SECONDS)) {
            executor.shutdownNow();                  // force l'arrêt au bout de 10
s
        }
    }
}
```

Le contrôleur, lui, ne voit rien de tout cela : il appelle simplement `vehiculeService.ajouterVehicule(v)`.

Attendre un résultat (facultatif). `submit(...)` renvoie un `Future`. Son `get()` attend la fin de la tâche et renvoie son résultat :

```
Future<Path> pdf = executor.submit(() -> genererEtatPdf());
// ... le menu continue ...
Path fichier = pdf.get();   // bloque jusqu'à ce que le PDF soit prêt
```

Quel service choisir ?

Création	Comportement	Usage dans le TP
`Executors.newSingleThreadExecutor()`	1 thread, tâches dans l'ordre	écritures en base
`Executors.newFixedThreadPool(2)`	2 threads en parallèle	PDF, courriel, exports

Pièges fréquents.

- **Oublier `shutdown()`** : le programme ne se termine pas, car le thread du service reste en vie.
- **Exceptions silencieuses** : une erreur dans une tâche n'apparaît nulle part. Mettez toujours un `try/catch` dans la tâche et journalisez l'erreur.
- **Lire le clavier dans une tâche** : le `Scanner` reste dans le menu, sur le thread principal.

Annexe C — Chiffrer un fichier avec AES

L'idée. AES est un algorithme de chiffrement **symétrique** : la **même clé** sert à chiffrer et à déchiffrer, comme un cadenas qui s'ouvre et se ferme avec une seule clé. Sans la clé, le fichier n'est qu'une suite de caractères illisibles.

À ne pas confondre avec BCrypt (question 4) : BCrypt fabrique une empreinte **à sens unique**, on ne retrouve jamais le mot de passe. AES, au contraire, permet de **retrouver** le texte d'origine.

Le vocabulaire.

Mot	Ce que c'est	Dans le TP
Clé	Le secret qui chiffre et déchiffre (256 bits = 32 octets)	Fichier `aes.key`, à garder secret
IV (vecteur d'initialisation)	Un nombre aléatoire, nouveau à chaque chiffrement. Il évite que deux exports identiques donnent le même résultat	12 octets, écrits au début du fichier chiffré
Mode GCM	La façon d'appliquer AES. GCM chiffre **et** détecte si le fichier a été modifié	`AES/GCM/NoPadding`
Base64	Transforme des octets en lettres et chiffres lisibles	Pour écrire la clé et le fichier chiffré en texte

Les étapes.

1. Générer une clé une seule fois, puis la sauvegarder dans `aes.key`.
2. Pour chiffrer : tirer un IV au hasard, chiffrer le texte, écrire IV + texte chiffré dans le fichier.

3. Pour déchiffrer : relire le fichier, séparer l'IV du reste, déchiffrer avec la même clé.

La classe complète. Elle n'utilise que le JDK, aucune librairie à ajouter.

```
package util;

import javax.crypto.*;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.security.GeneralSecurityException;
import java.security.SecureRandom;
import java.util.Base64;

public final class ChiffrementAES {

    private static final String ALGO = "AES/GCM/NoPadding";
    private static final int TAILLE_IV = 12;     // en octets
    private static final int TAILLE_TAG = 128;   // contrôle d'intégrité, en bits

    private ChiffrementAES() { }

    /** Génère une nouvelle clé AES de 256 bits. */
    public static SecretKey genererCle() throws GeneralSecurityException {
        KeyGenerator generateur = KeyGenerator.getInstance("AES");
        generateur.init(256);
        return generateur.generateKey();
    }

    /** Écrit la clé dans un fichier, en Base64. */
    public static void sauvegarderCle(SecretKey cle, Path fichier) throws IOException
{
        Files.writeString(fichier,
Base64.getEncoder().encodeToString(cle.getEncoded()));
    }

    /** Relit une clé sauvegardée avec sauvegarderCle. */
    public static SecretKey chargerCle(Path fichier) throws IOException {
        byte[] octets = Base64.getDecoder().decode(Files.readString(fichier).trim());
        return new SecretKeySpec(octets, "AES");
    }

    /** Chiffre un texte et l'écrit dans un fichier texte (Base64). */
    public static void chiffrer(String texte, SecretKey cle, Path fichier)
            throws GeneralSecurityException, IOException {
        byte[] iv = new byte[TAILLE_IV];
        new SecureRandom().nextBytes(iv);                       // IV aléatoire

        Cipher cipher = Cipher.getInstance(ALGO);
        cipher.init(Cipher.ENCRYPT_MODE, cle, new GCMParameterSpec(TAILLE_TAG, iv));
        byte[] chiffre = cipher.doFinal(texte.getBytes(StandardCharsets.UTF_8));
```

```
        ByteBuffer tampon = ByteBuffer.allocate(iv.length + chiffre.length);
        tampon.put(iv).put(chiffre);                            // IV d'abord, puis
les données
        Files.writeString(fichier,
Base64.getEncoder().encodeToString(tampon.array()));
    }

    /** Relit un fichier chiffré et renvoie le texte en clair. */
    public static String dechiffrer(SecretKey cle, Path fichier)
            throws GeneralSecurityException, IOException {
        byte[] contenu =
Base64.getDecoder().decode(Files.readString(fichier).trim());

        Cipher cipher = Cipher.getInstance(ALGO);
        cipher.init(Cipher.DECRYPT_MODE, cle,
                    new GCMParameterSpec(TAILLE_TAG, contenu, 0, TAILLE_IV)); // les
12 premiers octets = IV
        byte[] clair = cipher.doFinal(contenu, TAILLE_IV, contenu.length -
TAILLE_IV);
        return new String(clair, StandardCharsets.UTF_8);
    }
}
```

Exemple d'utilisation.

```
Path fichierCle = Path.of("aes.key");
SecretKey cle;
if (Files.exists(fichierCle)) {
    cle = ChiffrementAES.chargerCle(fichierCle);          // lancements suivants
} else {
    cle = ChiffrementAES.genererCle();                    // premier lancement
    ChiffrementAES.sauvegarderCle(cle, fichierCle);
}

String texte = "matricule;nom;prenom\n204518;Le Goff;Morgane\n";
ChiffrementAES.chiffrer(texte, cle, Path.of("salaries.enc"));

System.out.println(ChiffrementAES.dechiffrer(cle, Path.of("salaries.enc")));
```

Pièges fréquents.

- **Écrire seulement `"AES"`** dans `Cipher.getInstance(...)` : Java choisit alors le mode ECB, qui n'est pas sûr. Indiquez toujours `AES/GCM/NoPadding`.
- **Réutiliser le même IV** : il doit être tiré au hasard à chaque chiffrement. C'est pour cela qu'on l'enregistre dans le fichier, il n'est pas secret.
- **Perdre la clé** : les fichiers deviennent impossibles à relire, il n'existe aucun moyen de les récupérer.
- **`AEADBadTagException` au déchiffrement** : la clé n'est pas la bonne, ou le fichier a été modifié.

Annexe D

Ecran de Connexion

```
==========================================
   GESTION DE LA FLOTTE DE VÉHICULES
==========================================
Login        : gestionnaire
Mot de passe : ****
Identifiants incorrects. Tentatives restantes : 2

Login        : gestionnaire
Mot de passe : ********
Connexion réussie. Bienvenue, gestionnaire.
```

Apres 3 echecs:

```
Identifiants incorrects. Nombre maximal de tentatives atteint.
Fermeture de l'application.
```

Menu Principal

```
============== MENU PRINCIPAL ==============
 VÉHICULES
   1. Afficher la flotte
   2. Ajouter un véhicule
   3. Retirer un véhicule
   4. Rechercher par immatriculation
   5. Trier la flotte
 SALARIÉS ET RÉSERVATIONS
   6. Créer un salarié
   7. Enregistrer un départ
   8. Enregistrer un retour
   9. Véhicules en déplacement
```

```
  10. Historique d'un salarié
 ÉTATS ET EXPORTS
  11. Générer l'état PDF et l'envoyer par courriel
  12. Exporter salariés et réservations (AES)
  13. Lire un fichier exporté
   0. Quitter
==========================================
```

Votre choix :

Afficher la Flotte

Votre choix : 1

```
FLOTTE — 5 véhicules
+-----------+----------+-----------+------------+-----------+----------------+
| Immat.    | Marque   | Modèle    | Mise circ. | Km        | État           |
+-----------+----------+-----------+------------+-----------+----------------+
| AB-123-CD | Peugeot  | Partner   | 12/09/2022 |    12 450 | Disponible     |
| EF-751-GH | Citroën  | Berlingo  | 03/02/2020 |    30 120 | Disponible     |
| GH-482-KL | Renault  | Kangoo    | 15/03/2021 |    48 210 | En déplacement |
| JK-306-MN | Renault  | Clio      | 21/06/2018 |    95 870 | Disponible     |
| PQ-914-RS | Dacia    | Jogger    | 08/11/2023 |     8 330 | En déplacement |
+-----------+----------+-----------+------------+-----------+----------------+
```

Ajouter un véhicule

```
Votre choix : 2

AJOUT D'UN VÉHICULE
Immatriculation (AA-123-AA)        : tu-208-vw
Marque                             : Toyota
Modèle                             : Yaris
Mise en circulation (jj/mm/aaaa)   : 14/04/2024
Kilométrage                        : 5200
```

```
Véhicule TU-208-VW ajouté. Enregistrement en base en cours…
```

Rechercher et Retirer un véhicule

```
Votre choix : 3

Immatriculation du véhicule à retirer : GH-482-KL

Refusé : le véhicule GH-482-KL est en déplacement. Enregistrez son
retour d'abord.


Immatriculation du véhicule à retirer : JK-306-MN

Confirmer le retrait de JK-306-MN Renault Clio ? (o/n) : o

Véhicule JK-306-MN retiré de la flotte.


Votre choix : 4

Immatriculation : gh-482-kl


GH-482-KL  Renault Kangoo  (mis en circulation le 15/03/2021, 48 210
km)

État : en déplacement depuis le 27/10/2026 09:00 avec Morgane Le Goff
(204518)
```

Trier la Flotte

```
Votre choix : 5


  1. Par marque, puis modèle
  2. Par date de mise en circulation (plus ancien d'abord)
  3. Par kilométrage décroissant

Tri : 3


FLOTTE TRIÉE PAR KILOMÉTRAGE DÉCROISSANT
  JK-306-MN  Renault Clio       95 870 km
  GH-482-KL  Renault Kangoo     48 210 km
```

```
  EF-751-GH  Citroën Berlingo   30 120 km
  AB-123-CD  Peugeot Partner    12 450 km
  PQ-914-RS  Dacia Jogger        8 330 km
```

Créer un salarié

```
Votre choix : 6

CRÉATION D'UN SALARIÉ
Matricule (6 chiffres) : 204518
Nom                    : le goff
Prénom                 : morgane

Récapitulatif
  Matricule : 204518
  Nom       : Le Goff
  Prénom    : Morgane
Confirmer la création ? (o/n) : o
Salarié 204518 — Morgane Le Goff créé.
```

Enregistrer un depart

```
Votre choix : 7

DÉPART D'UN VÉHICULE
Matricule du salarié : 204518
Salarié : Morgane Le Goff
Immatriculation      : AB-123-CD
Refusé : Morgane Le Goff a déjà un véhicule en déplacement (GH-482-
KL).
```

Enregistrer un retour

```
Votre choix : 8

RETOUR D'UN VÉHICULE
Immatriculation          : GH-482-KL
Kilométrage au compteur  : 47900
Refusé : kilométrage inférieur au compteur actuel (48 210 km).

Kilométrage au compteur  : 48475
Retour enregistré : GH-482-KL, 265 km parcourus. Nouveau kilométrage :
48 475 km.
```

Véhicules en déplacement

```
Votre choix : 9

VÉHICULES EN DÉPLACEMENT — 2
+-----------+------------------+------------------+------------------+
| Immat.    | Véhicule         | Salarié          | Départ           |
+-----------+------------------+------------------+------------------+
| PQ-914-RS | Dacia Jogger     | Lucie Daniel     | 26/10/2026 08:15 |
| AB-123-CD | Peugeot Partner  | Yann Kerbrat     | 28/10/2026 19:42 |
+-----------+------------------+------------------+------------------+
```

Historique d’un salarié

```
Votre choix : 10
Matricule : 204518
```

```
HISTORIQUE — Morgane Le Goff (204518) — 3 réservations
  27/10/2026 09:00  →  27/10/2026 17:40   GH-482-KL  Renault Kangoo
  20/10/2026 08:15  →  20/10/2026 12:05   EF-751-GH  Citroën Berlingo
  06/10/2026 13:30  →  07/10/2026 18:00   AB-123-CD  Peugeot Partner
```

Etat PDF et courriel

```
Votre choix : 11
Génération de l'état PDF lancée en arrière-plan.

============== MENU PRINCIPAL ==============
...
Votre choix :
[PDF] État généré : etat-flotte.pdf (5 véhicules)
[Courriel] État envoyé à pascal.lamy5@ac-rennes.fr
```

Export chiffré avec AES

```
Votre choix : 12
Exports lancés en arrière-plan.
[Export] salaries.enc : 4 salariés exportés et chiffrés.
[Export] reservations.enc : 9 réservations exportées et chiffrées.
```

Lire un fichier AES

```
Votre choix : 13
Fichier à lire : reservations.enc

Contenu déchiffré de reservations.enc :
matricule;immatriculation;date_depart;date_retour;km_retour
204518;GH-482-KL;2026-10-27 09:00:00;2026-10-27 17:40:00;48475
117342;AB-123-CD;2026-10-28 19:42:00;
```

Annexe E — Spécificateurs de format

Les méthodes `System.out.printf(...)` et `String.format(...)` remplacent chaque spécificateur (commençant par `%`) par la valeur correspondante, dans l'ordre des arguments. Forme générale : `%[drapeaux][largeur][.précision]conversion`.

Conversions de base

Spécificateur	Signification	Exemple	Affichage
`%d`	Entier (int, long)	`printf("%d", 42)`	`42`
`%f`	Réel (6 décimales par défaut)	`printf("%f", 3.5)`	`3,500000`
`%.2f`	Réel avec 2 décimales (arrondi)	`printf("%.2f", 3.14159)`	`3,14`
`%e`	Réel en notation scientifique	`printf("%e", 1234.5)`	`1,234500e+03`
`%s`	Chaîne de caractères	`printf("%s", "clio")`	`clio`
`%c`	Caractère	`printf("%c", 'A')`	`A`
`%b`	Booléen	`printf("%b", true)`	`true`
`%x`	Entier en hexadécimal	`printf("%x", 255)`	`ff`
`%o`	Entier en octal	`printf("%o", 8)`	`10`
`%%`	Le caractère % lui-même	`printf("%d %%", 75)`	`75 %`
`%n`	Saut de ligne	`printf("Fin%n")`	`Fin ↵`

Largeur, précision et drapeaux

Spécificateur	Signification	Exemple	Affichage
`%5d`	Largeur 5, aligné à droite	`printf("%5d", 42)`	`···42`
`%-5d`	Largeur 5, aligné à gauche	`printf("%-5d|", 42)`	`42···|`
`%05d`	Largeur 5, complété par des 0	`printf("%05d", 42)`	`00042`

Spécificateur	Signification	Exemple	Affichage
`%8.2f`	Largeur 8, 2 décimales	`printf("%8.2f", 3.14159)`	`····3,14`
`%-10s`	Chaîne sur 10 caractères, à gauche	`printf("%-10s|", "clio")`	`clio·····|`
`%,d`	Séparateur de milliers	`printf("%,d", 1234567)`	`1 234 567`
`%+d`	Affiche toujours le signe	`printf("%+d", 42)`	`+42`

· représente un espace. Sur un système en français, les réels s'affichent avec une virgule (3,14) ; avec `Locale.US` *on obtient un point :* `String.format(Locale.US, "%.2f", x)`.